

Code Complexity Analysis for USEI11

Directed Line Upgrade Plan

Project: Logistics On Rails
Sprint: 3
User Story: USEI11 - Directed Line Upgrade Plan
Date: December 2024

Overview

This document analyzes the time complexity of all critical methods in the USEI11 implementation using Big-O notation.

Dataset: Belgian Railway Network

- V = 559 stations (vertices)
- E = 691 connections (edges)

Algorithms:

- Colored DFS (Cycle Detection) - ESINF06-Graph.pdf, slides 90-92
- Kahn's Algorithm (Topological Sort) - ESINF06-Graph.pdf, slide 116

Method Analysis

1. CycleDetection.findStationsInCycles()

Code:

```
public Set<V> findStationsInCycles(Graph<V, E> graph) {  
    CycleDetectionResult<V> result = detectCycles(graph); // O(V + E)  
  
    Set<V> stationsInCycles = new LinkedHashSet<>(); // O(1)  
  
    for (List<V> cycle : result.getCycles()) { // O(C)  
        stationsInCycles.addAll(cycle); // O(L)  
    }  
  
    return stationsInCycles; // O(1)  
}
```

Overall Complexity: $O(V + E)$

Deterministic: YES

2. CycleDetection.coloredDFS()

Code:

```

private boolean coloredDFS(Graph<V, E> graph,
                        V current,
                        Map<V, Color> color,
                        LinkedList<V> path,
                        List<List<V>> cycles) {

    color.put(current, Color.GRAY);           // O(1)
    path.addLast(current);                   // O(1)

    boolean foundCycle = false;              // O(1)

    Collection<V> adjVertices = graph.adjVertices(current); // O(1)
    if (adjVertices != null) {               // O(1)
        for (V adjacent : adjVertices) {     // O(deg(v))

            if (color.get(adjacent) == Color.GRAY) { // O(1)
                extractCycle(path, adjacent, cycles); // O(L)
                foundCycle = true;             // O(1)

            } else if (color.get(adjacent) == Color.WHITE) { // O(1)
                if (coloredDFS(graph, adjacent, color, path, cycles)) {
                    foundCycle = true;         // O(1)
                }
            }
        }
    }

    color.put(current, Color.BLACK);          // O(1)
    path.removeLast();                        // O(1)

    return foundCycle;                       // O(1)
}

```

Overall Complexity: $O(V + E)$

Deterministic: YES

3. CycleDetection.extractCycle()

Code:

```

private void extractCycle(LinkedList<V> path, V cycleStart,
                        List<List<V>> cycles) {

    List<V> cycle = new ArrayList<>();       // O(1)
    boolean recording = false;               // O(1)

    for (V vertex : path) {                  // O(|path|)
        if (vertex.equals(cycleStart)) {     // O(1)
            recording = true;                 // O(1)
        }
        if (recording) {                     // O(1)
            cycle.add(vertex);                // O(1)
        }
    }
    cycle.add(cycleStart);                    // O(1)

    cycles.add(cycle);                       // O(1)
}

```

Overall Complexity: $O(V)$

Deterministic: YES

4. TopologicalSort.kahn()

Code:

```
public List<V> kahn(Graph<V, E> graph) {
    List<V> topologicalOrder = new ArrayList<>();           // O(1)
    Queue<V> queue = new LinkedList<>();                  // O(1)
    Map<V, Integer> inDegree = new HashMap<>();            // O(1)

    for (V vertex : graph.vertices()) {                    // O(V)
        int degree = graph.inDegree(vertex);                // O(V)
        inDegree.put(vertex, degree);                       // O(1)

        if (degree == 0) {                                  // O(1)
            queue.add(vertex);                               // O(1)
        }
    }                                                        // Total: O(V²)

    int processedVertices = 0;                              // O(1)

    while (!queue.isEmpty()) {                              // O(V)
        V current = queue.poll();                           // O(1)
        topologicalOrder.add(current);                       // O(1)
        processedVertices++;                                // O(1)

        for (V adjacent : graph.adjVertices(current)) {     // O(E)
            int newInDegree = inDegree.get(adjacent) - 1;   // O(1)
            inDegree.put(adjacent, newInDegree);             // O(1)

            if (newInDegree == 0) {                          // O(1)
                queue.add(adjacent);                         // O(1)
            }
        }
    }

    if (processedVertices < graph.numVertices()) {         // O(1)
        throw new IllegalStateException(
            "Graph has cycles! Cannot perform topological sort.");
    }

    return topologicalOrder;                                // O(1)
}
```

Overall Complexity: $O(V^2)$

Note: The bottleneck is calculating in-degrees. `MapGraph.inDegree()` iterates through all vertices, making it $O(V)$ per call.

Deterministic: YES

5. UpgradePlanController.calculateUpgradeOrder()

Code:

```

public UpgradePlanResult calculateUpgradeOrder() {
    if (network == null) { // 0(1)
        throw new IllegalStateException("Network not loaded");
    }

    long startTime = System.currentTimeMillis(); // 0(1)

    Set<Station> stationsInCycles =
        cycleDetector.findStationsInCycles(network); // 0(V + E)

    if (!stationsInCycles.isEmpty()) { // 0(1)
        long elapsedTime = System.currentTimeMillis() - startTime;
        return UpgradePlanResult.withCycles(
            stationsInCycles,
            network.numVertices(),
            network.numEdges(),
            elapsedTime
        ); // 0(1)
    }

    List<Station> order = topologicalSort.kahn(network); // 0(V²)

    long elapsedTime = System.currentTimeMillis() - startTime;

    return UpgradePlanResult.withOrder(
        order,
        network.numVertices(),
        network.numEdges(),
        elapsedTime
    ); // 0(1)
}

```

Overall Complexity: $O(V^2)$

Deterministic: YES

6. BelgianNetworkLoader.loadNetwork()

Code:

```

public static Graph<Station, Connection> loadNetwork(
    String stationsPath, String linesPath) throws IOException {

    Map<String, Station> stationMap = loadStations(stationsPath); // 0(V)

    Graph<Station, Connection> graph = new MapGraph<>(true); // 0(1)

    for (Station station : stationMap.values()) { // 0(V)
        graph.addVertex(station); // 0(1)
    }

    int validLines = loadLines(linesPath, stationMap, graph); // 0(E)

    return graph; // 0(1)
}

```

Overall Complexity: $O(V + E)$

Deterministic: YES

7. BelgianNetworkLoader.loadStations()

Code:

```

private static Map<String, Station> loadStations(String filePath)
    throws IOException {
    Map<String, Station> stations = new HashMap<>();           // O(1)

    try (BufferedReader br = new BufferedReader(
        new FileReader(filePath))) {                          // O(1)

        String line = br.readLine();                          // O(1)

        while ((line = br.readLine()) != null) {              // O(V)
            line = line.trim();                                // O(L)
            if (line.isEmpty()) continue;                      // O(1)

            String[] parts = line.split(",");                  // O(L)
            if (parts.length >= 2) {                            // O(1)
                String id = parts[0].trim();                    // O(L)
                String name = parts[1].trim();                  // O(L)

                Station station = new Station(id, name);        // O(1)
                stations.put(id, station);                      // O(1)
            }
        }
    }

    return stations;                                           // O(1)
}

```

Overall Complexity: $O(V)$

Note: L is the average line length, treated as a constant.

Deterministic: YES

8. BelgianNetworkLoader.loadLines()

Code:

```
private static int loadLines(String filePath,
                             Map<String, Station> stationMap,
                             Graph<Station, Connection> graph)
    throws IOException {
    int validLines = 0; // O(1)

    try (BufferedReader br = new BufferedReader(
        new FileReader(filePath))) { // O(1)

        String line = br.readLine(); // O(1)

        while ((line = br.readLine()) != null) { // O(E)
            line = line.trim(); // O(L)
            if (line.isEmpty()) continue; // O(1)

            try {
                String[] parts = line.split(","); // O(L)
                if (parts.length >= 5) { // O(1)
                    String fromId = parts[0].trim(); // O(L)
                    String toId = parts[1].trim(); // O(L)
                    double distance = Double.parseDouble(
                        parts[2].trim()); // O(L)
                    int capacity = Integer.parseInt(
                        parts[3].trim()); // O(L)
                    double cost = Double.parseDouble(
                        parts[4].trim()); // O(L)

                    Station from = stationMap.get(fromId); // O(1)
                    Station to = stationMap.get(toId); // O(1)

                    if (from != null && to != null) { // O(1)
                        Connection conn = new Connection(
                            from, to, distance, capacity, cost); // O(1)
                        graph.addEdge(from, to, conn); // O(1)
                        validLines++; // O(1)
                    }
                }
            } catch (NumberFormatException e) { // O(1)
                // Error handling
            }
        }

        return validLines; // O(1)
    }
```

Overall Complexity: $O(E)$

Note: L is the average line length, treated as a constant.

Deterministic: YES

Summary Table

Method	Class	Time Complexity	Deterministic
findStationsInCycles()	CycleDetection	$O(V + E)$	YES
coloredDFS()	CycleDetection	$O(V + E)$	YES
extractCycle()	CycleDetection	$O(V)$	YES
kahn()	TopologicalSort	$O(V^2)$	YES

<code>calculateUpgradeOrder()</code> Method	<code>UpgradePlanController</code> Class	$O(V^2)$ Time Complexity	YES Deterministic
<code>loadNetwork()</code>	<code>BelgianNetworkLoader</code>	$O(V + E)$	YES
<code>loadStations()</code>	<code>BelgianNetworkLoader</code>	$O(V)$	YES
<code>loadLines()</code>	<code>BelgianNetworkLoader</code>	$O(E)$	YES

Where:

- V = 559 stations (vertices)
- E = 691 connections (edges)

Global Complexity

Overall USEI11 Complexity: $O(V^2)$

Breakdown:

1. Data Loading: $O(V + E)$
2. Cycle Detection: $O(V + E)$
3. Topological Sort (if no cycles): $O(V^2)$

Dominant Term: $O(V^2)$ from `TopologicalSort.kahn()`

Belgian Railway Dataset Performance:

- V = 559, E = 691

Determinism

Is USEI11 Deterministic? YES

All methods are deterministic:

- Same input always produces same output
- No random number generators
- No external system dependencies
- No multithreading
- Verified through extensive testing with Belgian Railway dataset